

PRINZIPIEN

Maurice Müller

Murphy's Law

deutsch: Murphys Gesetz

Definition

- "Anything that can go wrong will go wrong."
- "Alles, was schiefgehen kann, wird schiefgehen."
- Übertragung
 - erwarte Fehler
 - gestalte die Applikation möglichst robust

Geschichtlicher Hintergrund

- benannt nach Edward A. Murphy Jr. (US-Luftwaffeningenieur)
- entstand 1949 bei Raketenschlitten-Experimenten
- Auslöser: ein Assistent hatte Sensoren falsch verkabelt
- bekannt geworden durch John Stapp, dem Projektleiter, der diese Worte in einer späteren Pressekonferenz zitierte

Beispiel

Was kann da schon schiefgehen?

```
public int process(Data data) {  
    List<Integer> values = data.getDetails().getValues();  
    return values.stream().mapToInt(it -> it).sum();  
}
```

- NullPointerException wenn
 - *data* null ist
 - *data.getDetails()* null ist
 - einer der *Integer* null ist

Beispiel

Robustere Lösung

```
public int process(Data data) {  
    if (data == null || data.getDetails() == null) {  
        throw new IllegalArgumentException();  
    }  
    return values.stream()  
        .mapToInt(it -> it != null? it : 0)  
        .sum();  
}
```

Murphys Gesetz

- in dem Sinn kein Prinzip oder Gesetz
- Erinnerung, mit dem Schlimmsten zu rechnen

Information Hiding

deutsch: Geheimnisprinzip

Definition

- Verbergen von Implementierungsdetails hinter einer stabilen Schnittstelle
- Zugriff auf Daten und interne Logik wird kontrolliert
- Ziel: Module können unabhängig voneinander geändert werden, solange die Schnittstelle gleich bleibt
- Eng verwandt mit der Kapselung (Encapsulation)

Beispiel: Bankkonto

Direkter Zugriff (schlecht)

```
class BankAccount {  
    // not allowed to be less 0  
    public double balance;  
}  
  
// Usage  
account.balance = -500; // invalid numbers are possible
```

Beispiel: Bankkonto

Mit Information Hiding

```
class BankAccount {  
    private double balance;  
  
    void deposit(double amount) {  
        if (amount < 0)  
            throw new IllegalArgumentException();  
        balance += amount;  
    }  
  
    boolean draw(double amount) {  
        if (amount < 0 ||  
            balance - amount < 0) {  
            throw new IllegalArgumentException();  
        }  
        balance -= amount;  
    }  
}
```

- Konto kann nicht mehr überzogen werden
- mögliche Fehler (negative *amounts*) werden nicht verarbeitet

Beispiel: Einstellungen

- eine Möglichkeit wird gebraucht applikationsweite Einstellungen zu verwalten
- erster Wurf: statische *HashMap*

```
public static Map<String, String> settings = new HashMap<>();
```

- (-) kann von überall überschrieben werden
 - → wenigstens *final* machen
- (-) jede Einstellung kann von überall überschrieben werden
- (-) Einstellungen können ungültige Werte bekommen
- (-) Einstellungen können gelöscht werden
 - Gefahr einer *NullPointerException*
- (-) erschwerte Tests

Beispiel: Einstellungen

Lösung: Kapselung in eine eigene Klasse

```
class Configuration {  
    private final Map<String, String> settings = new HashMap<>();  
  
    Configuration() {  
        // load settings (e.g. from a file)  
    }  
  
    public String getSetting(String key) {  
        return settings.get(key);  
    }  
}
```

- niemand kann Einstellungen ändern
- niemand kann Einstellungen löschen
- verhindert die falsche Nutzung
- pro Test kann eine eigene *Configuration* instanziiert werden

Beispiel: Einstellungen

kompletter Austausch der Implementierung bei Bedarf möglich

```
class Configuration {  
    private final DBTable table = /*...*/;  
  
    public String getSetting(String key) {  
        return table.queryById(key);  
    }  
}
```

- ermöglicht Änderungen, ohne die Aufrufer anzupassen
- ermöglicht neue Anwendungsfälle ohne die Aufrufer anzupassen
 - z.B. könnten zur Laufzeit Einstellungen geändert werden, indem Änderungen in der Datenbank gemacht werden (fragwürdiges Verhalten, aber es würde gehen)

Information Hiding: Vorteile

- Aufrufer muss keine Details kennen
- Aufrufer kann keine Fehler machen
- Daten bleiben konsistent
- Code wird modularer
- verringert die Kopplung

DRY

DON'T REPEAT YOURSELF

Definition

- Code-Duplikation vermeiden
- kopierte Codestellen sind kopierte Fehler
 - Refactoring wird erschwert
- kopierter Code deutet auf falsche oder fehlende Abstraktion hin
- gilt auch für duplizierte Logik (unterschiedlicher Code löst das gleiche Problem)
 - falls unterschiedliche Lösungen Sinn machen → Strategy Pattern

Beispiel: DRY

Wo ist die Duplikation?

```
class PersonProcessor {  
    void processStudent(String name, int age) {  
        if (name == null || name.isEmpty()) {  
            System.out.println("Error: Name cannot be empty");  
            return;  
        }  
        System.out.println("Processing student: " + name + ", age: " + age);  
    }  
  
    void processTeacher(String name, int age) {  
        if (name == null || name.isEmpty()) {  
            System.out.println("Error: Name cannot be empty");  
            return;  
        }  
        System.out.println("Processing teacher: " + name + ", age: " + age);  
    }  
}
```

Beispiel: Lösung

```
class PersonProcessor {  
    private boolean validateName(String name) {  
        if (name == null || name.isEmpty()) {  
            System.out.println("Error: Name cannot be empty");  
            return false;  
        }  
        return true;  
    }  
  
    void processStudent(String name, int age) {  
        if(!validateName(name)) return;  
        System.out.println("Processing student: " + name + ", age: " + age);  
    }  
  
    void processTeacher(String name, int age) {  
        if(!validateName(name)) return;  
        System.out.println("Processing teacher: " + name + ", age: " + age);  
    }  
}
```

- erhöhte Lesbarkeit
- jeder Name wird überall gleich validiert
- Änderungen an der Namensvalidierung können zentral gemacht werden

Mögliche weitere Verbesserung: Exception

```
class PersonProcessor {  
    private boolean validateName(String name) {  
        if (name == null || name.isEmpty()) {  
            throw new RuntimeException("Invalid name: " + name);  
        }  
    }  
  
    void processStudent(String name, int age) {  
        validateName(name);  
        System.out.println("Processing student: " + name + ", age: " + age);  
    }  
  
    void processTeacher(String name, int age) {  
        validateName(name);  
        System.out.println("Processing teacher: " + name + ", age: " + age);  
    }  
}
```


DRY: allgemeines Vorgehen

- *3 strikes and you refactor*
 - 1. Mal wird die Logik neu implementiert
 - 2. Mal wird die Logik kopiert und angepasst
 - 3. Mal wird eine saubere Abstraktion erstellt
- Duplikationen auslagern in
 - eigene Methoden
 - eigene Klassen

KISS

KEEP IT SIMPLE, STUPID!

oder: keep it short and simple

Definition

- *einfach* ist oft besser als *kompliziert*
- Code muss verständlich sein und nicht die eigene "Genialität" ausdrücken
- Code wird häufiger gelesen als geschrieben
- Code wird von und für Menschen geschrieben

Beispiel: KISS

```
public class Fingers {  
  
    enum Finger {THUMB, INDEX, MIDDLE, RING, LITTLE}  
  
    private Finger[] fingers = {Finger.THUMB, Finger.INDEX,  
                                Finger.MIDDLE, Finger.RING, Finger.LITTLE};  
  
    public Finger fromNumber(int number) {  
        if(number < 0 || number > 4) {  
            throw new RuntimeException("Invalid number: " + number);  
        }  
        return fingers[number - 1];  
    }  
}
```


Beispiel: Lösung

```
public class Fingers {  
  
    enum Finger {THUMB, INDEX, MIDDLE, RING, LITTLE}  
  
    public Finger fromNumber(int number) {  
        return switch(number) {  
            case 0 -> Finger.THUMB;  
            case 1 -> Finger.INDEX;  
            case 2 -> Finger.MIDDLE;  
            case 3 -> Finger.RING;  
            case 4 -> Finger.LITTLE;  
        };  
    }  
}
```

Bei Enums eher auf *ordinal* / die Position verzichten.

```
public class Logs {  
    enum LogLevel {ERROR, INFO, TRACE}  
  
    public LogLevel fromNumber(int number) {  
        return LogLevel.values()[number];  
    }  
}
```

Möglicher Fehler bzgl. *ordinal*

```
public class Logs {  
    enum LogLevel {FATAL, ERROR, INFO, DEBUG, TRACE}  
  
    public LogLevel fromNumber(int number) {  
        return LogLevel.values()[number];  
    }  
}
```

- *ordinal* hat sich verschoben
 - kann zum Problem bei persistierten Daten werden
 - kann zum Problem bei serialisierten Daten werden
 - z.B. falls die Gegenseite (noch) nicht aktualisiert wurde

KISS

Ähnlich: Einfachheit vor Allgemeinverwendbarkeit

- allgemeinverwendbare Lösungen tendieren zu erhöhter Komplexität
- trotzdem sinnvolle Abstraktionen nutzen

Beispiel: Allgemeinverwendbarkeit

```
public interface Validator<T> {  
    boolean validate(T value);  
}  
  
public class UserNameValidator {  
    private List<Validator<String>> validators = new ArrayList<>();  
  
    public void addValidator(Validator<String> validator) {  
        validators.add(validator);  
    }  
  
    public boolean isValid(String username) {  
        return validators.stream().allMatch(v -> v.validate(username));  
    }  
}
```

Beispiel: Einfachheit

```
public class UserNameValidator {  
    public boolean isValid(String username) {  
        return username != null  
            && username.length() >= 3  
            && username.length() <= 20;  
    }  
}
```

YAGNI

You ain't gonna need it!

Du wirst es nicht brauchen!

Definition

- Funktionalität erst implementieren, wenn man sie braucht
 - strikt: sogar dann, wenn es absehbar ist
- nicht implementierte Funktionalität kostet nichts:
 - keine Tests
 - keine Bugs
 - keine Verwirrung

Beispiel: YAGNI

User soll persistiert werden

```
record User(  
    UUID id,  
    String name,  
    String email  
) {}
```

```

class UserRepository {
    private DBConnection db = DBConnection.primary();
    private List<UserActivityLog> activityLogs = new ArrayList<>();
    private UserCache cache = new UserCache();

    public void addUser(UUID userId, String name, String email) {
        User user = new User(userId, name, email);
        user.setCreatedTimestamp(System.currentTimeMillis());
        user.setRole(UserRole.USER);
        user.setDefaultPermission(PermissionLevel.BASIC);
        cache.addToCache(userId, user);
        activityLogs.add(new UserActivityLog(userId, "USER_CREATED"));
        db.putUser(userId, user);
    }
}

```

für die Aufgabe "User persistieren" unnötig komplex

Beispiel: Lösung

```
class UserRepository {  
    private DBConnection db = DBConnection.primary();  
  
    public void addUser(UUID userId, String name, String email) {  
        User user = new User(userId, name, email);  
        db.putUser(userId, user);  
    }  
}
```

Beispiel: CSV-Export

Anforderung: Export von Bestelldaten als CSV

```
interface ExportService {
    void export(List<Order> orders, ExportConfig config);
}

class UniversalExportService implements ExportService {
    public void export(List<Order> orders, ExportConfig config) {
        if (config.format() == Format.CSV) {
            /* ... */
        }
        if (config.format() == Format.JSON) {
            /* ... */
        }
        if (config.format() == Format.HTML) {
            /* ... */
        }
    }
}
```


Beispiel: CSV-Export

```
class CsvExportService {  
    public void export(List<Order> orders) {  
        // Einfache CSV Generierung  
    }  
}
```

YAGNI: Warum funktioniert es?

- je später Funktionalität implementiert wird
 - umso mehr ist bekannt
 - umso mehr Erfahrung hat die umsetzende Person
- evtl. macht eine andere Funktionalität die ursprüngliche Idee überflüssig
- evtl. gibt es neue Libraries/Frameworks, die genau das übernehmen

Premature Optimization

deutsch: Voreilige Optimierung

Premature optimization is the root of all evil.

— Donald Knuth

Definition

- Optimierung von Code, bevor eine Optimierung notwendig ist
- führt oft zu:
 - komplexerem Code
 - schlechterer Wartbarkeit
 - mehr Fehlern
 - geringem oder gar keinem Performance-Gewinn

Die 90/10-Regel

- 90% der Ausführungszeit wird in nur 10% des Codes verbraucht
- Optimierung der restlichen 90% des Codes hat fast keinen Effekt auf die Gesamtperformance
- häufig sind temporäre schlechte Ausführungszeiten verschmerzbar

Regeln zur Optimierung

1. **Don't** (Mach es nicht)
2. **Not yet** (Mach es später)
3. **Measure** (Analyse durch Messen)

Performancemessungen

Echte Performance-Messungen sind schwierig durchzuführen.

- Profiler nutzen
- Wiederholungen eher in der Größenordnung ab 100.000 Durchläufen anstatt 1.000 oder weniger
- Problem, eine Maschine frei von Nebeneffekten zu bekommen
- Problem, dass z.B. die JVM während des Betriebs Optimierungen macht
- Zeitmessungen an sich sind schwierig (v.a. bei sehr schnellen Code-Schnipseln)

Beispiel: Cache

```
public class PermissionService {  
    private Map<String, List<String>> cache = new ConcurrentHashMap<>();  
  
    public List<String> getPermissions(String userId) {  
        return cache.computeIfAbsent(userId, id ->  
            database.findPermissionsByUserId(id)  
        );  
    }  
  
    public void evictCache(String userId) {  
        cache.remove(userId);  
    }  
}
```

- Speicherverbrauch steigt
- Cache-Invalidierung ist schwierig (wann ist der Cache veraltet?)
- Komplexität durch Thread-Sicherheit (ConcurrentHashMap)

SDK entwickelt sich weiter

Alt	Aktuell	Warum
<pre>new StringBuilder() .append("User ") .append(name) .append(" hat die ID ") .append(id) .toString();</pre>	<pre>"User " + name + " hat die ID " + id;</pre>	seit JDK9: <code>invokedynamic</code> zum Aufruf von JDK-Bibliotheksfunktionen
try/catch vermeiden weil 'teuer'	try/catch nutzen	try/catch wurde schon im Jahr 2000 optimiert
Exception-Werfen vermeiden weil 'teuer'	Exceptions werfen (außerhalb des Kontrollflusses)	Exceptions sind im Vergleich teuer, aber außerhalb des Kontrollflusses ist das in der Regel vernachlässigbar

Regeln zur Optimierung

1. **Don't** (Mach es nicht)
2. **Not yet** (Mach es später)
3. **Measure** (Analyse durch Messen)

Law Of Demeter

deutsch: Gesetz von Demeter

Definition

- "Don't talk to strangers" (Sprich nicht mit Fremden)
- ein Objekt sollte nur mit Objekten in seiner unmittelbaren Umgebung kommunizieren
- eine Methode *m* einer Klasse *C* darf nur folgende Objekte aufrufen
 - Methoden von *C*
 - Parameter von *m*
 - Objekte, die in *m* erzeugt wurden
 - direkte Komponenten von *C* (Instanzvariablen)

Beispiel: Method Chaining

Verletzung des LoD (Train Wreck)

```
double amount = person.getWallet().getMoney().getAmount();
```

- eine Null-Reference reicht für eine NullPointerException
- Wissen über die verzweigte Struktur ist notwendig
- wenn sich die Struktur ändert, muss sich der Aufrufer ändern

Beispiel: Method Chaining

Versuch, der Verhinderung des *Train Wrecks*

```
double amount = 0.0;
var wallet = person.getWallet();
if (wallet != null) {
    var money = wallet.getMoney();
    if (money != null) {
        amount = money.getAmount();
    }
}
```

- bläht den eigentlichen Code auf (Boiler Plate)
- jeder muss das so nachprogrammieren, der die gleiche Information braucht

Beispiel: Method Chaining

Lösung: Delegation

```
class Person {  
    double getAvailableMoney() {  
        if (wallet == null) return 0.0;  
        var money = wallet.getMoney();  
        if (money == null) return 0.0;  
        return money.getAmount();  
    }  
}
```

- jeder kann die Funktionalität wiederverwenden
- unterstützt Information Expert
- (-) LoD immer noch verletzt

Beispiel: Method Chaining

```
class Person {  
    double getAvailableMoney() {  
        if (wallet == null) return 0.0;  
        return wallet.getAvailableMoney();  
    }  
}  
  
class Wallet {  
    double getAvailableMoney() {  
        if (money == null) return 0.0;  
        return money.getAmount();  
    }  
}
```


Law Of Demeter: Vorteile

- lose Kopplung
- erhöhte Wiederverwendbarkeit
- unterstützt Information Expert
- (-) u.U. viele Delegationen
- (-) u.U. viele Methoden, die ausschließlich delegieren

Konvention vor Konfiguration

auch: Convention over Configuration, Configuration By Exception

Definition

- Frameworks / Module mit sinnvollen Voreinstellungen / Konventionen ausliefern
- häufig wird nur ein Teil gebraucht → der Rest soll 'einfach funktionieren'
- (-) Mehraufwand um gute Defaults zu überlegen
- Beispiel: **Spring**

Beispiel

Wie kann diese Methode sinnvoll vorkonfiguriert werden?

```
class Filter {  
    // null = match all  
    public final String lastName;  
    public final Integer age;  
    /* ... */  
}  
  
public List<User> fetchUsers(Filter filter) {  
    Query<User> query = new Query<>();  
    query.matches(filter.lastName, User::getLastName);  
    //...  
    return query.execute();  
}
```

Lösung: Methode überladen

```
public List<User> fetchUsers() {  
    return fetchUsers(Filter.All);  
}
```


Beispiel (fortgesetzt)

Vorkonfigurierte Filter

```
class UserFilter implements Filter {  
  
    static final Filter All = new UserFilter(...);  
    static final Filter Adult = new UserFilter(...);  
    static final Filter Child = new UserFilter(...);  
  
    // [...]  
}
```

Konvention vor Konfiguration

der ideale Methodenaufruf

```
Magic.doNiceStuff();
```

- keine Instanziierung
- keine Parameter
- trotzdem passiert etwas brauchbares und sinnvolles

Prinzip der minimalen Verwunderung

englisch: Principle of Least Astonishment

- das Offensichtliche tun
 - d.h., das tun, was man erwarten würde
- z.B. getDate() liefert ein Datum zurück und kein Alter

Beispiel: Minimale Verwunderung

```
List<Integer> filterEvenNumbers(List<Integer> data) {  
    List<Integer> result = new ArrayList<>();  
    for (Integer num : data) {  
        if (num % 2 == 0) {  
            result.add(num);  
        }  
    }  
    for (Integer num : this.cache) {  
        if (num % 2 == 0 && !result.contains(num)) {  
            result.add(num);  
        }  
    }  
    return result;  
}
```


Beispiel: Lösung

```
public List<Integer> filterEvenNumbers(List<Integer> data) {  
    return data.stream().filter(it -> it %2 == 0).toList();  
}  
public List<Integer> combineWithCacheAndFilterEven(List<Integer> data) {  
    List<Integer> combined = new ArrayList<>(data);  
    combined.addAll(this.cache);  
    return filterEvenNumbers(combined);  
}
```

Prinzip solider Annahmen

englisch: Principal Of Solid Assumptions

- nicht auf Vermutungen aufbauen, sondern auf Wissen
- wenn Vermutungen dann gut dokumentieren
 - **besser:** Sicherstellen der Vermutungen

Beispiel: Prinzip solider Annahmen

```
public double calculateAverage(List<Integer> numbers) {  
    int sum = 0;  
    for (int number : numbers) {  
        sum += number;  
    }  
    return sum / numbers.size();  
}
```

- Annahme, dass *numbers* nie null ist
- Annahme, dass *numbers* nie leer ist

Beispiel: Lösung

Möglichkeit 1: Dokumentieren

```
// the whole module allows only lists with at least one number  
// so we assume that we also have at least one number  
public double calculateAverage(List<Integer> numbers) {  
    int sum = 0;  
    for (int number : numbers) {  
        sum += number;  
    }  
    return sum / numbers.size();  
}
```


Beispiel: Lösung

Möglichkeit 2: Sicherstellen

```
public double calculateAverage(List<Integer> numbers) {  
    if(numbers == null || numbers.isEmpty()) {  
        System.err.println("Got an invalid list.");  
        return 0.0; // Alternative: Exception  
    }  
    int sum = 0;  
    for (int number : numbers) {  
        sum += number;  
    }  
    return sum / numbers.size();  
}
```

Beispiel: E-Mail-Adresse

```
class EmailAddress {
    EmailAddress(String plainAddress) {
        // [validate address or exception]
        this.plainAddress = plainAddress;
    }
}

class EmailService {
    void sendEmail(EmailAddress address, String content) {
        // [send email]
    }
}
```

- die E-Mail-Adresse muss nicht validiert werden